

Integration Guide X-platform

Version 1.3

Table of Contents

1. Overview	2
1.1. Versioning	2
1.2. Reference Manual	2
1.3. Exploring the API	2
2. Using JSON-RPC	3
2.1. Endpoints and Interfaces	3
2.1.1. Request header	4
2.1.2. Request payload	4
2.1.3. Response	5
2.1.4. Failure response	5
2.2. Examples	5
2.3. JSON format	6
2.3.1. Primitive types	7
2.3.2. Composite types	8
2.3.3. Example	10
3. Authentication process	13
3.1. BeginSession	13
3.2. RefreshSession	14
3.3. EndSession	14
3.4. GetTokenInfo	15
3.5. GetUsers	15
3.6. SetPassword	16
4. Fundamentals	17
4.1. Anatomy of a unit	17
4.2. Sources	17
4.3. Signal flow	19
4.4. Outputs	20
5. Monitoring	21
5.1. Alarms	21
5.2. Chassis model	22
6. IP Gateway	24
6.1. IP Interface and Physical ports	24
6.2. IP input and input mode	26
6.3. Ip Output and Output modes	27
6.4. Seamless inputs and Cloned outputs	30
7. ASI	32
7.1. ASI physical ports	32
7.2. ASI inputs	33
7.3. ASI outputs	34

Revision History

Revision	Date	Name	Comment
1.0	26. May 2020	Steffen Olsen	Initial Release
1.1	28. January 2021	Michael Brandt	Extend Authentication and IP Gateway chapter
1.2	29. January 2021	Michael Brandt	Add chapter for ASI module
1.3	02. December 2021	Michael Brandt	Update postman collections

1. Overview

This document describes the automation interface on the Appear X platform. It describes the general message protocol and the transfer mechanism used and includes messages for each protocol. Detailed description of every command and its parameters is provided in corresponding reference manual.

An Appear unit on the X platform provides a remote automation interface using HTTPS and JSON-RPC protocols. Interaction is initiated by the client by sending a request message to the server (unit), which is always answered with a response message (even if empty one) or a failure message. Message structure is the same regardless of protocol used.

1.1. Versioning

The remote interface tags commands and messages with version information and expects it to be specified when making a request. As with message format, version level of a message exchange is determined by the first message and does not change until the exchange is completed (i.e. response is of the same version as the request). Units provide backward compatibility and accept commands of all previous versions if possible.

1.2. Reference Manual

Reference Manual for the remote interface of given version describes structure and format of each command and message. Parameters are listed and described in a format-agnostic way, how to interpret them is explained in the specific sections later in this document.

1.3. Exploring the API

There are various tool that makes it possible to explore the X platform APIs, if access to a physical unit is possible.

Browser Dev tools

Various browsers comes with Development tools that makes it possible to inspect the HTTP traffic between the browser and the server. Since the integrated UI of the unit is solely relying on the JSON-RPC API, the development tools network tools will give full visibility of the RPC commands the UI is using.

Postman

Postman is an open source API development tool which is made to build, test and modify APIs. Some of the JSON-RPC commands used in this document is available as a file which can be imported into Postman. To use it you need to follow these steps:

1. Download Postman
2. Make sure to disable SSH certificate validation. This is done in Settings→General→SSL Certificate Validation
3. Extract the postman.zip file
4. Import postman_environment.json and postman collection files
5. Set the following environment variables to match your setup:
 - CHASSIS_IP – Set the IP address of a X-unit
 - USERNAME – User account, typically admin
 - PASSWORD – Password for the user account
6. Inside each collection set the variables to match your setup:
 - BOARD_SLOT – Set the slot position of the board you would like to test

2. Using JSON-RPC

2.1. Endpoints and Interfaces

Generally speaking, there are three different types of RPC endpoints on the Appear X platform. Each endpoint provides access to one or more versioned interfaces. These interfaces, and which endpoints name to use, are described in detail by their respective reference manuals.

MMI

Provides access to system-wide functionality like authentication, user management and general system overview.

```
https://ADDRESS/mmi/...
```

Board

Provides access to product specific functionality like input and output configuration.

```
https://ADDRESS/board/SLOT/...①
```

^① SLOT needs to be specified in hexadecimal form, i.e slot 10 is specified as **A**

Service

Product RPC service that encapsulates product functionality across multiple boards. Note that the service name is prefixed with **service_**.

```
https://ADDRESS/mmi/service_NAME/...
```

e.g. service_mmi, service_encoderpool, service_decoderpool



Please note that all endpoints only accept HTTPS connections.

The JSON-RPC interface is using the JSON-RPC 2.0 specification and can be accessed via the endpoints described above, with the extension **/api/jsonrpc**. The URL for the JSON-RPC requests is depending on the endpoint and interface:

```
https://ADDRESS/mmi/api/jsonrpc
```

```
https://ADDRESS/board/SLOT/api/jsonrpc
```

```
https://ADDRESS/mmi/service_NAME/api/jsonrpc
```

Each **endpoint** supports one or more **versioned** interfaces, each containing one or more **modules** where the **commands** are defined. The version of the **interface** also defines the underlying version of the **module** and **command**. Hierarchy of the terms:

- **EndPoint**
 - **Interface**
 - **Module**

▪ Command

Hierarchy example of MMI endpoint

```
https://192.168.1.100/mmi/api/jsonrpc
  mmi:2.16
    alarms
      GetActiveAlarms
      GetAlarmsHistory
    authentication
      BeginSession
      EndSession
```

Hierarchy example of BOARD endpoint

```
https://192.168.1.100/board/<SL0T>/api/jsonrpc
  board:2.16
    services
      GetInputServices
      GetOutputServices
    about
      GetCardAbout
  ipGateway:1.15
    input
      GetInputs
      SetInputs
    output
      GetOutputs
      SetOutputs
```

2.1.1. Request header

RPC commands are issued by making HTTP requests over HTTPS (HTTP with TLS). The HTTP request made to a unit requires the following headers are set in the HTTP header.

```
Authorization: Bearer TOKEN ①
```

① Authentication token

All RPC commands needs to be authenticated, with some exceptions. The authentication procedure is described in detail in [Authentication process](#)

2.1.2. Request payload

The HTTP payload is a serialized JSON object where the following properties need to be specified:

```
{
  "jsonrpc": "2.0",
  "id" : REQUEST_ID, ①
  "method" : "<interface>:<version>/<module>/<command>" ②
  "params" : REQUEST_DATA ③
```

```
}
```

- ① An identifier established by the Client that MUST contain a string or number. Will be part of the corresponding response data.
- ② Please note that interface name and version (effectively a namespace) must be part of the method object in the JSON-RPC request message. The structure of the method string is:
`"method" : "<interface>:<version>/<module>/<command>"`
- ③ Optional. Request data according to the specification in the reference manual, see [JSON format](#) section for details.

2.1.3. Response

The response is a serialized JSON object with the following structure:

```
{
  "jsonrpc": "2.0",
  "id": REQUEST_ID, ①
  "result": RESPONSE_DATA, ②
}
```

- ① User defined id specified in the request
- ② Response data according to the specification in the reference manual, see [JSON format](#) section for details.

2.1.4. Failure response

In the case of failure, the following response will be given

```
{
  "jsonrpc": "2.0",
  "id": REQUEST_ID, ①
  "error": { ②
    "code": -32602,
    "data": {
      "message": "failure description"
    }
  }
}
```

- ① User defined id specified in the request
- ② Error information

2.2. Examples

For this example let's assume there is an interface **interface** version **1.0**, our unit's IP address is **192.168.1.100** and we are communicating with the MMI board on slot 1. Endpoints for the JSON-RPC requests are:

```
https://192.168.1.100/mmi/api/jsonrpc
```

```
https://192.168.1.100/board/1/api/jsonrpc
```

```
https://192.168.1.100/mmi/service_mmi/api/jsonrpc
```

A JSON-RPC request for command **command** from module **module** of this interface could look like this:

Request

```
{
  "jsonrpc": "2.0",
  "method": "interface:1.0/module/command",
  "params": {
    "queryId": 123
  },
  "id": 3
}
```

Response

```
{
  "jsonrpc": "2.0",
  "result": {
    "text": "foo"
  },
  "id": 3
}
```

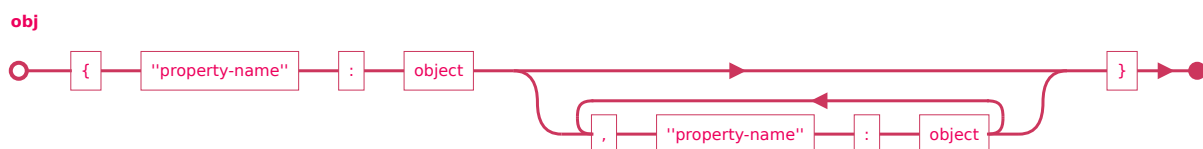
Failure response

```
{
  "jsonrpc": "2.0",
  "error": {
    "code": -32602,
    "data": {
      "message": "failure description"
    }
  },
  "id": 3
}
```

2.3. JSON format

Parameters in the reference manual are described in terms of types. This part describes how those are represented in a JSON message.

JSON is a language-independent syntax for serializing objects. A JSON object consists of a set of properties with values. Property names must be double-quoted strings. The structure of a JSON object is show in the diagram below



Where an object can be either of:

- Primitive types:
 - String
 - Enum
 - Bigint
 - Int
 - Float
 - Bool
 - Uuid
- Composite types:
 - Struct
 - Variant
 - Optional
 - List
 - Set
 - Map

2.3.1. Primitive types

String

String type values are encoded as string surrounded by `"`

```
"property-name" : "string-value"
```

UUID

UUID value encoded as a string

```
"property-name" : "12bc7140-e72e-499e-82d1-7ecdb3799bcb"
```

Enum

A string restricted to declared values

```
"property-name" : "ENUMVAL"
```

Bigint

A string with number. Used for 64bit integers

```
"property-name" : "123"
```

Int

An integer, 32bit

```
"property-name" : 123
```

Float

A number

```
"property-name" : 123.4
```

Bool

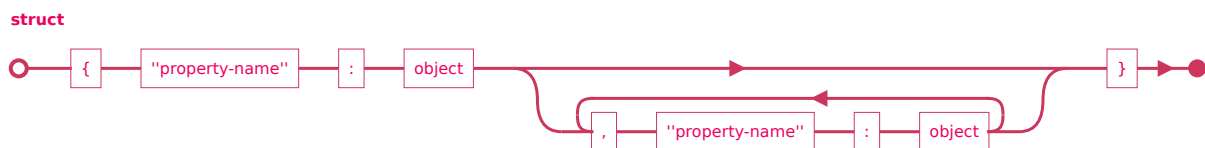
A boolean, possible values are either `true` or `false`

```
"property-name" : true
```

2.3.2. Composite types

Struct

A struct object is a composition of several parameters in the following form:

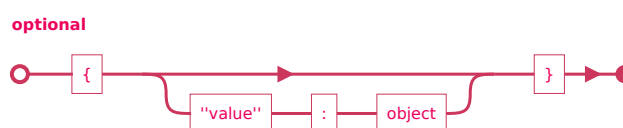


Where `obj` can be either a primitive or composite type.

```
{
  "property-name1" : 23,
  "property-name2" : "a value",
  "property-name3" : {
    "property-name" : "something"
  }
}
```

Optional

The optional type is an option type, with added optionality state – present or not present. Presence is specified adding a struct object with a **value** key and the specified type.



```
{
```

```

"property-name1" : { "value" : 32}, ①
"property-name2" : {} ②
}

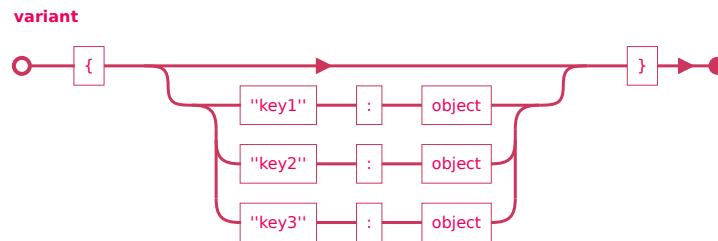
```

① Specifying an optional to a integer type of value 32

② Specifying not present.

Variant

An object with one or none matching member keys. Like an optional, except value can be one of the specified types.



```

{
  "property-name1" : { "keyA" : "Test"}, ①
  "property-name2" : { "keyB" : 32}, ②
  "property-name3" : {} ③
}

```

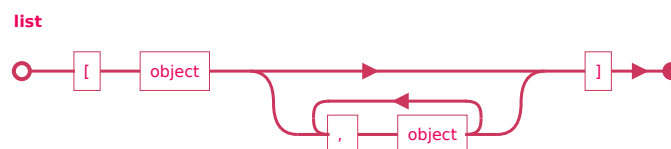
① Specifying a variant to "keyA", which is specified to be of type string, to a string with value "Test"

② Specifying a variant to "keyB", which is specified to be of type int to an int with value 32

③ Specifying not present.

List

An array of objects of the specified type



```

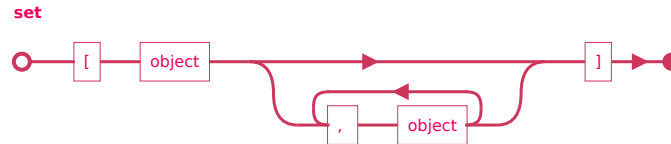
{
  "property-name1" : [1,2,3,4,5] ①
}

```

① Specifying a list of integers

Set

An array of objects of the specified type

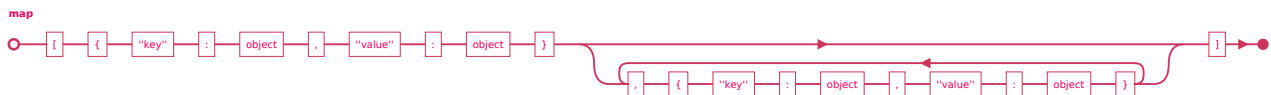


```
{
  "property-name1" : [1,2,3,4,5] ①
}
```

① Specifying a set of integers

Map

An array of objects with unique keys and values of the specified subtypes



```
{
  "property-name1" : [
    { "key" : "keyA" , "value" : [1,2,3,4,5] } , ①
    { "key" : "keyB" , "value" : [6,7,8,9,0] }
  ]
}
```

① Specifying a map with two entries. The key is specified to be of string type and the value objects as a set of integers

2.3.3. Example

Given the following type declaration:

X MODULE_NAME

X.Y Command Reference

X.Y.1 GetSomeStatus

- message **GetSomeStatus.request**
- message **GetSomeStatus.response**

X.Y.2 Type Reference

X.Y.2.1 GetSomeStatus.request

slot	int
details	optional Details

X.Y.2.2 GetSomeStatus.response

data	map from UUID to StatusInfo
------	---

StatusInfo

Table 1. struct

id	int
status	list of int

SomeEnum

enum

- VALUE_A - Description
- VALUE_B - Description
- VALUE_C - Description

Details

Table 2. struct

includeBufferA	SomeEnum
detailLevel	variant DetailLevel

DetailLevel

Table 3. variant

minimal	int
medium	string
maximum	bigint

It is possible to make the following request:

Request

```
{
  "jsonrpc": "2.0",
  "id" : 2,
  "method" : "ENDPOINT_NAME:ENDPOINT_VERSION/MODULE_NAME/GetSomeStatus",
  "slot" : 1,
  "params" : {
    "details" : {
      "value" : { ①
        "includeBufferA" : "VALUE_B", ②
        "detailLevel" : { ③
          "maximum" : "23232"
        }
      }
    }
  }
}
```

```
}
```

- ① Specifying an optional **Details**, by specifying a **Details** object, which is a **struct**, with the **value** property.
- ② Specifying enum value **VALUE_B**
- ③ Specifying **medium** for the **DetailLevel** variant.

could generate the following reply:

Response

```
{
  "jsonrpc": "2.0",
  "id" : 2,
  "result" : {
    "data" : [ ①
      { "key" : "65571fb1-5e35-4fd4-b1fa-b75511c1e326" , "value" : { "id": 1, "status" :
[2,3,4,5,6] } },
      { "key" : "5106ca3e-ac20-441e-8a4f-72adf7826227" , "value" : { "id": 2, "status" :
[1,2,3,4,5] } }
    ]
  }
}
```

- ① Map of **UUID** to **StatusInfo**

3. Authentication process

This section describes the authentication procedure on the Appear X platform.



Factory setting

A unit arriving from the factory will have a default user with the user name 'Admin' and an empty password. The token returned by `BeginSession` is in this case restricted to only allow calling the `SetPassword` method. The Password needs to be reset before any other RPC commands will be accepted.

In order to use most of the commands on the remote interface, the user must provide a valid access token as a proof that he is authorized for the operation. An access token can be obtained and managed by the authentication RPC interface as described in `BeginSession`.



To use the token, it needs to be included in the HTTP header in the following form:

```
Authorization: "Bearer <TOKEN>"
```

Access tokens are valid only for a limited time and will not be accepted by the Appear unit once expired. When that happens the user needs to sign in again to get a new token. An alternative is refreshing the session while the current token is still valid.



The following sections describe available authentication RPC methods. Examples are provided for the JSON-RPC protocol. The MMI endpoint has to be used for all authentication requests:

```
https://ADDRESS/mmi/api/jsonrpc
```

3.1. BeginSession

Generates a new access token if proper credentials are provided and returns it in the response message. The HTTPS header for this request does not contain the "Authorization" field.

Request

```
{
  "jsonrpc": "2.0",
  "method": "mmi:1.0/authentication/BeginSession",
  "params": {
    "local": {
      "username": "user",
      "password": "hello"
    }
  }
}
```

```
    },  
    "id": "login"  
  }  
}
```

Response

```
{
  "jsonrpc": "2.0",
  "result": {
    "accessToken":
    "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG91IiwiaWF0Ij0iYWRtaW4iOnRydWV9.TJVA950rM7E2cBab30RMhrHDcEfxjoYZgeFONFh7HgQ" ①
  },
  "id": "login"
}
```

① Token to be included in the HTTP header

3.2. RefreshSession

Generates a new access token if the user is already authenticated and provides a valid access token in the header. A new access token is returned in the response message as in the SignIn method.

Request

```
{
  "jsonrpc": "2.0",
  "method": "mmi:1.0/authentication/RefreshSession",
  "params": {},
  "id": "refresh"
}
```

3.3. EndSession

Ends the current session. The user is regarded as logged out.

Request

```
{
  "jsonrpc": "2.0",
  "method": "mmi:1.0/authentication/EndSession",
  "params": {},
  "id": "logout"
}
```



LDAP

When using LDAP each `BeginSession` call will result in the LDAP server will be involved in authenticating the user. To avoid unnecessary queries to the LDAP server it is important to use the `RefreshSession` to renew tokens

3.4. GetTokenInfo

Provides the user with more information about an access token, returned by `BeginSession`. This method does not require authentication, the access token in question is provided as a parameter in the request message.

Request

```
{
  "jsonrpc": "2.0",
  "id": "GetTokenInfo",
  "method": "{{PKG_NAME}}:{{API_VERSION}}/authentication/GetTokenInfo",
  "params": {
    "accessToken": "{{ACCESS_TOKEN}}"
  }
}
```

Response

```
{
  "jsonrpc": "2.0",
  "id": "GetTokenInfo",
  "result": {
    "username": "admin",
    "expires": 599,
    "valid": true
  }
}
```

3.5. GetUsers

Retrieves a list of all users in the system that are visible to the current user.

Request

```
{
  "jsonrpc": "2.0",
  "id": "GetUsers",
  "method": "{{PKG_NAME}}:{{API_VERSION}}/authentication/GetUsers",
  "params": {}
}
```

Response

```
{
  "jsonrpc": "2.0",
  "id": "GetUsers",
  "result": {
    "data": [
      {
        "id": "cf4dcf3f-48fc-5235-ba28-8b57fdbd01e6", ①
        "username": "admin",
        "groups": {"main": "37af3f9d-4f93-42a9-8212-92f2b24baad3"},
      }
    ]
  }
}
```

```
        "details": {"name": "", "email": ""},
        "block": {},
        "passwordExpiration": {"neverExpires": {} }
    }
}
}
```

① User UUID to be used in **SetPassword** method

3.6. SetPassword

Replaces the current password for the specified user id with a new one.

Request

```
{
  "jsonrpc": "2.0",
  "method": "mmi:1.0/authentication/SetPassword",
  "Params": {
    "userId": "cf4dcf3f-48fc-6235-ba28-9b57fdbd01e6",
    "password": "hello",
  },
  "id": "setpw"
}
```

4. Fundamentals

4.1. Anatomy of a unit

The unit is designed with reliability and flexibility in mind. It consists of a chassis in which a number of hot-swappable modules can be installed. The chassis can be configured to host interface and processing cards according to the customer's requirements.

A unit can be equipped with up to 2 switch modules, each providing a backplane network which is used for data-transfer and inter-card communication. Units installed with dual switch modules will thus provide two communication paths internally. If one of the switch modules fails, the other communication path is still active and communication and data-transfer between the remaining cards will not be affected (seamless switching is applied internally).

Modules typically have features that fall into one of the following categories:

Input Module

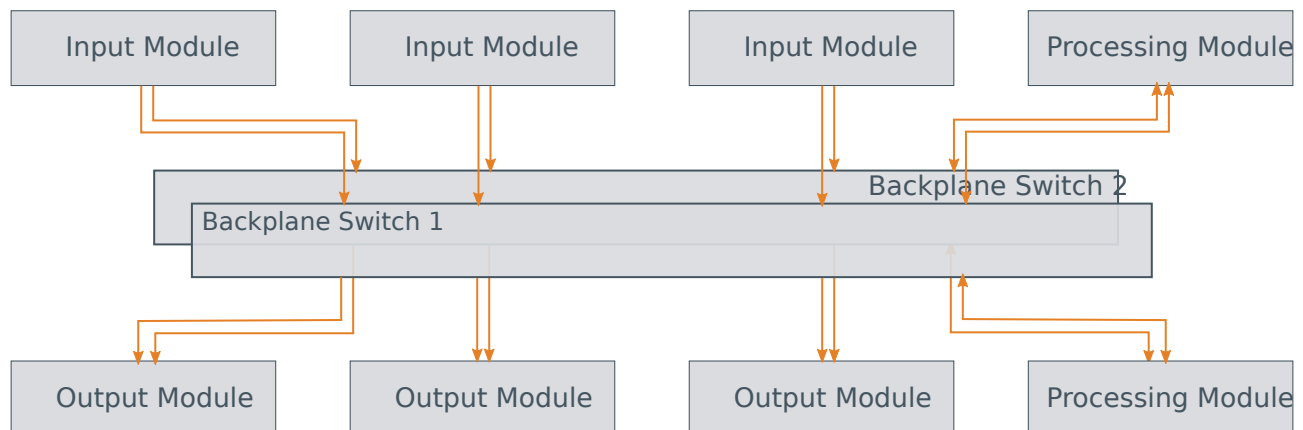
Input modules are capable of receiving sources from data networks, depending on the type of module. All input sources received can be routed to any of the other modules in the unit.

Processing Module

Processing modules process data routed from another modules (input or processing) and make the processed data stream available for routing to any other module in the unit.

Output module

Output modules generate output data streams which content can be taken from any other module in the unit.



Some modules carry both input and output capabilities simultaneously, and can be both an input module and output module in this regard.

4.2. Sources

Both input and processing modules will make data sources available for all other modules in the unit, they will publish possible input sources for all other modules. Processing modules will publish their processed sources to this reference system, making these available for the other modules. This is done through an internal reference system which contains all available sources in the unit.

It is possible to query this reference module by issuing a `GetInputServicesPage` (or `GetInputServices`) **command** to the `services module`, which is available on all boards.

The request will return a hierarchical data structure of `FlowSourceInfo` with `ChildSourceInfo`, which can be routed to Processing and Output modules by using product specific rpc requests, see their respective reference manuals for more information.

In this document, the top-level `FlowSourceInfo` is referred to as an `Input`, and the `FlowSourceInfo` children sources can be of the following types:

Table 4. Source types

Source type	ChildSourceInfo nodeType	Description
MPEG service	<code>Flow::FlowSource::ServiceSource::</code>	Identifies a MPEG/DVB TS defined service.
Auto-first service	<code>Flow::FlowSource::DvbSource::</code>	When an <code>Auto-first</code> service is mapped to a Processing or Output module, the unit will automatically resolve one of the MPEG/DVB services present on the Input.
EMM	<code>Flow::FlowSource::EmmSource::</code>	Identifies an EMM component.

An example response to a `GetInputServices` request is listed below. Some of the properties in the request has been removed to shorten the example.

Request

```
{
  "jsonrpc": "2.0",
  "method": "{{PKG_NAME}}:{{API_VERSION}}/services/GetInputServices",
  "params": {
    "query": {}
  },
  "id": "GetInputServices"
}
```

Response

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "data": [{
      "key": "bc925598-9c33-4b71-a3fa-7acb948cc647", ① ②
      "value": {
        "name": "239.250.4.1:1234",
        "label": "", ③
        "nodeType": "Flow::FlowSource::IpInputProxy:",
        "owner": "1", ④
        "isPresent": true,
        "sources": [{ ⑤
          "key": "e44f1889-ca00-5fce-b283-76e169224ef0", ⑥
          "value": {
            "name": "VH1 Classic",
            "nodeType": "Flow::FlowSource::ServiceSource:", ⑦
            "owner": "1", ④
            "isPresent": true ⑧
          }
        }
      ]
    }
  ]
}
```

```

    }, {
      "key": "15858edf-0161-5e7e-ac82-07292e2535c9", ⑥
      "value": {
        "name": "Auto First Service",
        "nodeType": "Flow::FlowSource::DvbSource::", ⑦
        "owner": "1",
        "isPresent": true
      }
    }, {
      "key": "909ff2a1-de71-5cde-b2fc-2fda9af81563", ⑥
      "value": {
        "name": "EMM (CAS 22020)",
        "nodeType": "Flow::FlowSource::EmmSource::", ⑦
        "owner": "12",
        "isPresent": true
      }
    }
  ]
}
}
}

```

- ① UUID of the Input, will typically match the UUID used when configuring
- ② This is the top level **FlowSourceInfo** referred to as the **Input**
- ③ User defined label
- ④ **owner** identifies the slot of the **Input**
- ⑤ Map **UUID** of **ChildSourceInfo** sources available on this input
- ⑥ **UUID** of **ChildSource**
- ⑦ NodeType identifies the type of source, see **Source types**
- ⑧ Defines whether source is currently present

Interface cards offer functionality to generate input and output to the unit, while processing cards are able to process data.

All sources are addressed by their UUID.

4.3. Signal flow

Sources for output flows needs to be explicitly mapped through the needed modules to meet the requirements for the output. This is done by mapping the source from the Input module where it originates to a possible processing module or Output module. If Processing is involved, the **corresponding** source published by the Processing module needs to be mapped to the next Processing module or output module. This mapping is done through the respective `product(module) rpc` methods, and referring to available sources as described in the **Sources** section.



- ① UUID (configuration ID) of Input
- ② Source UUID published by the Input Module.
- ③ On the input stage of the Descrambler module, the UUID in (2) is referenced.
- ④ The Descrambler module publishes the processed (descrambled) source in the reference system.

- ⑤ On the input stage of the Scrambler module, the UUID of the source in (4) is referenced.
- ⑥ The Scrambler module publishes the processed (scrambled) source in the reference system
- ⑦ The output configuration on (8) references the UUID of the source published in (6) to add the scrambled service to an output.
- ⑧ The Output configuration UUID (configuration ID)

Querying the signal flow

It is possible to query signal flow by requesting `GetServiceDependencies` method of the `service` module. The response will return a list of all sources in the unit, and which source each respective source depends on. For the case in the diagram above, the response will list:

- UUID (4) depends on UUID (2)
- UUID (6) depends on UUID (4)

4.4. Outputs

It is possible to get a list of outputs and which sources they contain by querying `GetOutputServices` method on the `service` module. This returns a list of sources per output UUID.

5. Monitoring

5.1. Alarms

Alarm information for all modules in the unit is available through a request to `getActiveAlarms` and `getAlarmsHistory`. The reply to both queries are identical, and an example is given below.

```
{
  "jsonrpc": "2.0",
  "id": "GetActiveAlarms",
  "method": "{{PKG_NAME}}:{{API_VERSION}}/alarms/GetActiveAlarms",
  "params": {
    "query": {}
  }
}
```

```
{
  "jsonrpc": "2.0",
  "id": 16,
  "result": {
    "data": [{
      "alarmId": "interface/no_link",①
      "alarmName": "No Link",
      "alarmDescription": "No link on dataport",
      "severity": "MAJOR",
      "details": "No link on dataport",
      "configObjectId": "20a021e7-0e36-580f-952f-b0656d2702e6",②
      "configObjectType": "physicalports:1.7/PhysicalPort",③
      "configObjectDescription": "Slot: 1 Interface: D2",④
      "configObjectSlot": 1,
      "configObjectLogin": {
        "slot": 1
      },
      "configObjectHandler": "IP Interfaces",
      "configObjectLabel": "",
      "timeSet": "1585029420",⑤
      "timeClear": {},⑥
      "position": 4,
      "filtered": false
    }],
    "sessionId": "2ddb4c2f-370b-4ecc-a341-4bb63429bd64"
  }
}
```

- ① Alarm Id of alarm. The individual alarm IDs are described in the Alarm description document.
- ② UUID used when configuring the entity that generates the alarm.
- ③ Type of configuration, maps to RPC `method`
- ④ User presentable description of the failed entity.
- ⑤ Time when the alarm was raised.
- ⑥ Time when the alarm was raised. If empty, the alarm is still active.

5.2. Chassis model

The **Chassis model** is an abstraction model of how the unit is configured. The model consists of nodes and relations between nodes. Each node can have a set of traits which gives information about each node. For a list of possible traits, refer to the reference manual.

Each node can have a Config UUID defined. This makes it possible to map data from other RPC's which expose Config UUID, like `getInputServices`, `getOutputServices` and `getActiveAlarms`.

Request

```
{
  "jsonrpc": "2.0",
  "id": "GetGraph",
  "method": "{{PKG_NAME}}:{{API_VERSION}}/chassisModel/GetGraph",
  "params": {}
}
```

Response

```
{
  "jsonrpc": "2.0",
  "id": 4,
  "result": {
    "graph": {
      "nodes": [
        {
          "key": 1, ①
          "value": [
            {
              "key": "c9edf7c4-2797-442a-85b0-6fcfb6ca27e0", ②
              "value": {
                "id": "c9edf7c4-2797-442a-85b0-6fcfb6ca27e0", ②
                "configId": {
                  "value": "c9edf7c4-2797-442a-85b0-6fcfb6ca27e0" ③
                },
                "display_name": "239.250.1.1:1234 (Test Label)", ④
                "card": {}, ⑤
                "connector": {},
                "ipInterface": {},
                "input": { ⑥
                  "value": {}
                },
                "asi": {},
                "ip": {
                  "value": {}
                }
              }
            },
            {}
          ]
        },
        {}
      ],
      "relations": [
        {

```



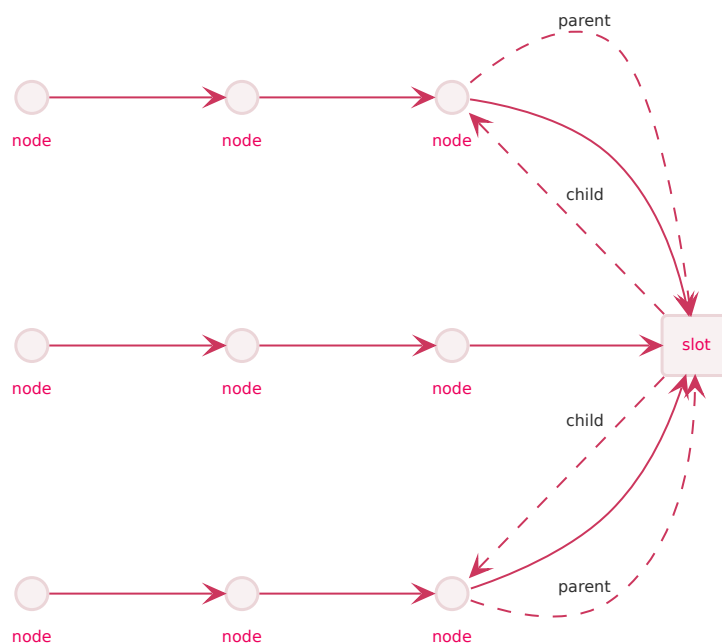
```

    "key": 1, ⑦
    "value": [
      {
        "key": "c9edf7c4-2797-442a-85b0-6fcfb6ca27e0", ⑧
        "value": {
          "parent": { ⑨
            "value": "9b81fb30-2d99-57bf-a9c7-0047002f240d"
          },
          "children": [] ⑩
        }
      },
      {}
    ]
  }
}

```

- ① Subgraph is for slot 1.
- ② Chassis model UUID.
- ③ Optional, Config UUID. Is not necessary identical to Chassis model UUID. Can be used with `getInputServices`, `getActiveAlarms` etc which also includes Config UUID.
- ④ Human readable name.
- ⑤ Empty trait, this node does not have the "card" trait.
- ⑥ Non-empty trait, this node has the "input" trait.
- ⑦ Relations for slot 1.
- ⑧ Relations for node with this Chassis model UUID.
- ⑨ Parent node for this node.
- ⑩ Children nodes for this node.

With the nodes and relations, it is possible to build a model of how the unit is configured.



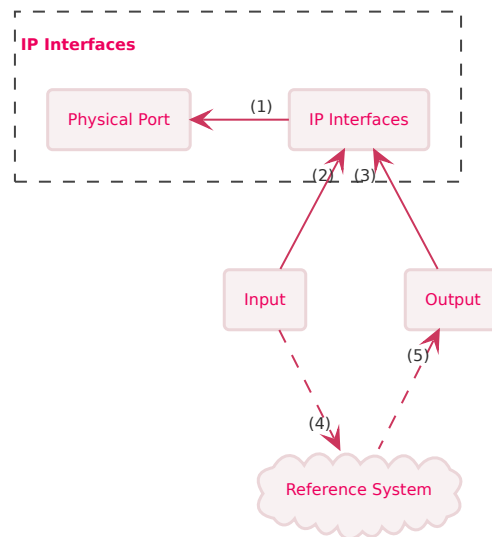
6. IP Gateway

The IP gateway module is a module which offers both input and output functionality simultaneously. Both Input and output streams are linked to an IP interface, which needs to be configured before any Inputs or Outputs can be configured. All configuration are done through the `board` endpoint and the `ipGateway` interface.

For a basic configuration of an IP Gateway module, following steps should be performed

1. Configure the physical IP ports needed, by using the `physicalports` module.
2. Configure the IP interfaces needed. These interfaces are configured through the `ipinterface` module on each IP Gateway board. IP interfaces are linked to a `PhysicalPort`
3. Configure possible IP inputs using `input` module.
4. Configure possible IP outputs using `output` module.

The conceptual model of the IP gateway is illustrated below:



- ① An Ip interface is linked to a Physical port by UUID
- ② An Ip input defines which IP interface it will use by linking to an IP Interface UUID
- ③ An Ip output defines which IP interface it will use by linking to an IP Interface UUID
- ④ Input module will analyze the received data-stream and publish available sources to the reference system as described in [Sources](#)
- ⑤ An Output will address sources in the reference system to generate an output data stream.

6.1. IP Interface and Physical ports

The IP gateway module has several Physical IP connectors, depending on the type of board installed. When configuring an IP gateway module's input and outputs, it is required that IP interfaces are correctly set up.

Table 5. Interface name mapping

Interface name	Interface Physical Label
ni6	CTRL
ni0	D1
ni1	D2

Interface name	Interface Physical Label
ni4	D3
ni5	D4

Request

```
{
  "jsonrpc": "2.0",
  "method": "{{PKG_NAME}}:{{API_VERSION}}/ipinterface/GetIpInterfaces",
  "params": {},
  "id": "GetIpInterfaces"
}
```

Response

```
{
  "jsonrpc": "2.0",
  "id": 4,
  "result": {
    "data": [
      {
        "key": "9b81fb30-2d99-57bf-a9c7-0047002f240d", ①
        "value": {
          "enabled": true,
          "label": "",
          "switchbackThreshold": {},
          "physicalPortId": "20a021e7-0e36-580f-952f-b0656d2702e6", ②
          "name": "ni0", ③
          "vlan": 0,
          "ipv4": {
            "value": {
              "ipAddress": "10.10.80.100",
              "gateway": "10.10.80.1",
              "netmask": 24
            }
          },
          "ipv6": {}
        }
      },
      {
        "key": "1eb515dd-e5af-5415-84cd-79fb0032909a", ①
        "value": { }
      },
      {
        "key": "3c8b61b2-abb3-59be-936b-9bb2e58a8ffc", ①
        "value": { }
      }
    ],
    ...
  }
}
```

- ① UUID of IpInterface. Will be used to address which IP interface to use when using `SetInputs` and `SetOutputs`
- ② UUID of the PhysicalPort where this IP Interface is defined.
- ③ Name of interface, see [Interface name mapping](#) for mapping rules.

6.2. IP input and input mode

Ip Inputs are maintained through the input **module**. New inputs are defined with the **SetInputs** command where the configuration is split into two parts; transport and analysis. Transport defines the relevant protocol depending on the network type while analysis configuration consists of settings describing how the data should be interpreted by the system.

The Analysis mode selected will also affect the type of mapping (to other modules) that is supported by the system. There are two main modes; **TS** and **RTP**. **TS** mode will make the system analyze the incoming stream metadata for information about the content in the stream, and populate services in the reference system as explained in [Sources](#). **RTP** will not analyze any content. Please see the table below

Table 6. Input modes

	Analyze mode	Description
TS	InputAnalyzeMpeg	Analyze input stream as a MPEG-TS.
TS	InputAnalyzeDvb	Analyze input stream as a MPEG-TS and DVB tables
TS	InputAnalyzeAtsc	Analyze input stream as a MPEG-TS and ATSC tables
RTP	InputRtp	Will not analyze any stream content. Can only be mapped as a raw RTP stream.

Request

```
{
  "jsonrpc": "2.0",
  "method": "{{PKG_NAME}}:{{API_VERSION}}/input/GetInputs",
  "params": {},
  "id": "GetInputs"
}
```

Response

```
{
  "jsonrpc": "2.0",
  "id": 5,
  "result": {
    "data": [
      {
        "key": "bc925598-9c33-4b71-a3fa-7acb948cc647", ①
        "value": {
          "transportSettings": {
            "udp": {
              "input": {
                "single": {
                  "interfaceId": "9b81fb30-2d99-57bf-a9c7-0047002f240d", ②
                  "destination": {
                    "address": "239.250.4.1",

```

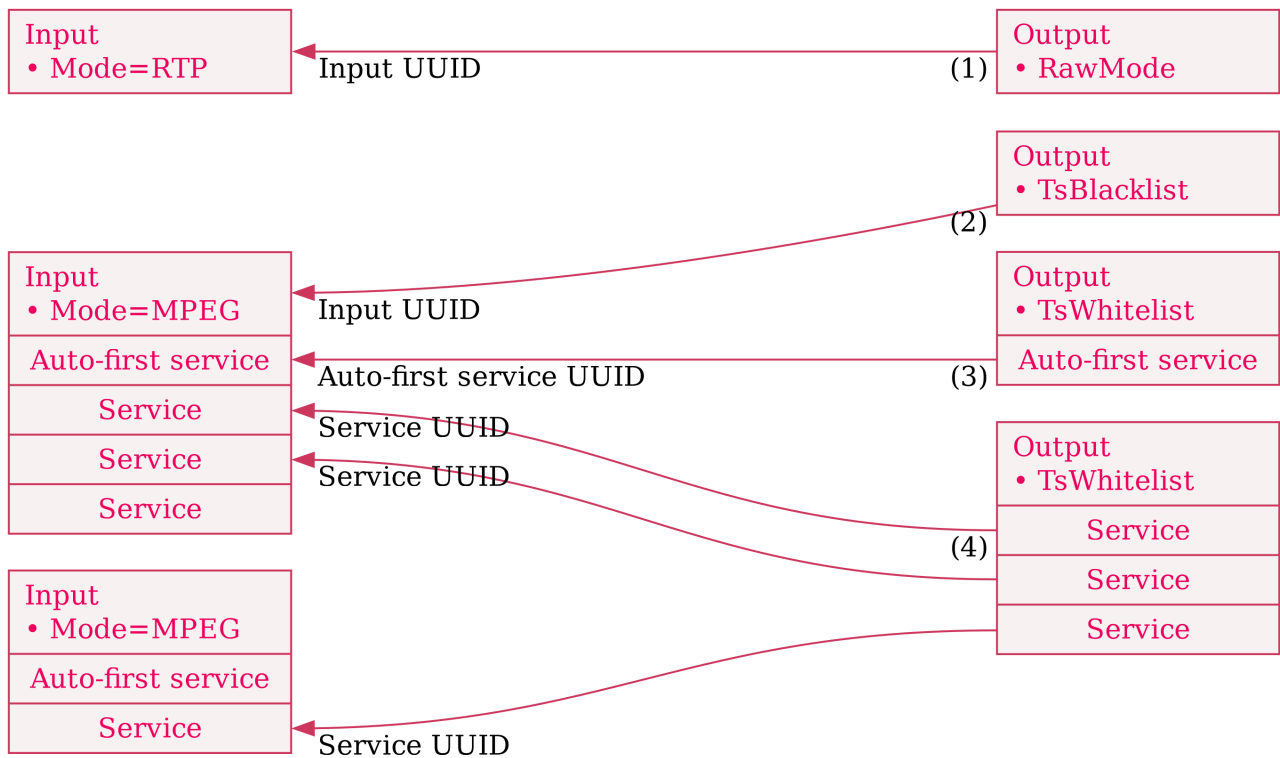



Figure 1. Example setup of dependencies between Input modes and Output modes.

- ① Output in TsBlacklist mode, and input in RTP mode. This will map the entire RTP stream from the input and preserve the IP headers.
- ② Output in TsBlacklist mode, and input in TS mode. This will map the entire input TS stream (including NULL packets), and no PSI/SI will be regenerated.
- ③ Output in TsWhitelist mode, and producing an output TS. The source for this output is an Auto-first service from an input, by referencing Auto-first service UUID as described in the [Sources](#) section.
- ④ Output in TsWhitelist mode, and producing an output TS. The sources in this TS are mapped from two different Inputs, by referencing Service UUIDs as described in the [Sources](#) section.

Request

```
{
  "jsonrpc": "2.0",
  "method": "{PKG_NAME}:{API_VERSION}/output/GetOutputs",
  "params": {},
  "id": "GetOutputs"
}
```

Response

```
{
  "jsonrpc": "2.0",
  "id": "test",
  "result": {
    "data": [
      {
        "key": "7009b27a-8f42-4257-ab2c-4b6f0486e84b", ①
        "value": {
```

```

    "label": "TS Mapping",
    "transportSettings": {
      "udp": {
        "output": {
          "single": {
            "interfaceId": "9b81fb30-2d99-57bf-a9c7-0047002f240d", ②
            "destination": {
              "address": "239.80.225.1",
              "port": 1234
            }
          }
        }
      }
    },
    "outputSettings": {
      "rawMode": {③
        "source": "c9edf7c4-2797-442a-85b0-6fcfb6ca27e0" ④
      }
    }
  },
  {
    "key": "e3c89b47-95bc-46c3-b638-aaabde1e4030",
    "value": {
      "label": "Service Mapping",
      "transportSettings": {
        "udp": {
          "output": {
            "single": {
              "interfaceId": "9b81fb30-2d99-57bf-a9c7-0047002f240d",
              "destination": {
                "address": "239.80.225.2",
                "port": 1234
              }
            }
          }
        }
      },
      "outputSettings": {
        "dvbMode": {
          "source": {
            "multiplex": [⑤
              {
                "service": {
                  "source": "5f7a751c-aa0a-5d59-be1c-6677bc435c52"
                }
              },
              {
                "service": {
                  "source": "db241d42-4d76-58fb-b28d-4a2e45b9f84c"
                }
              }
            ]
          }
        }
      }
    }
  }
}

```

```

    }
  }
}

```

- ① Configuration UUID
- ② UUID of IP interface where output will be streamed.
- ③ Output mode, as described in [Output modes](#)
- ④ Input UUID, as this is a TS mapping.
- ⑤ This output is in TsWhitelist mode, and Service sources are defined. These source UUIDs can be replaced on existing outputs to switch between services.

Supported types of `multiplex` sources in TsWhitelist mode:

- DVB `service` / ATSC `channel` / MPEG `program`
- `emm`
- `externalPid` (Source UUID has to be the Auto-first service UUID of the input to map the PID from)

See [Source types](#) for more information.

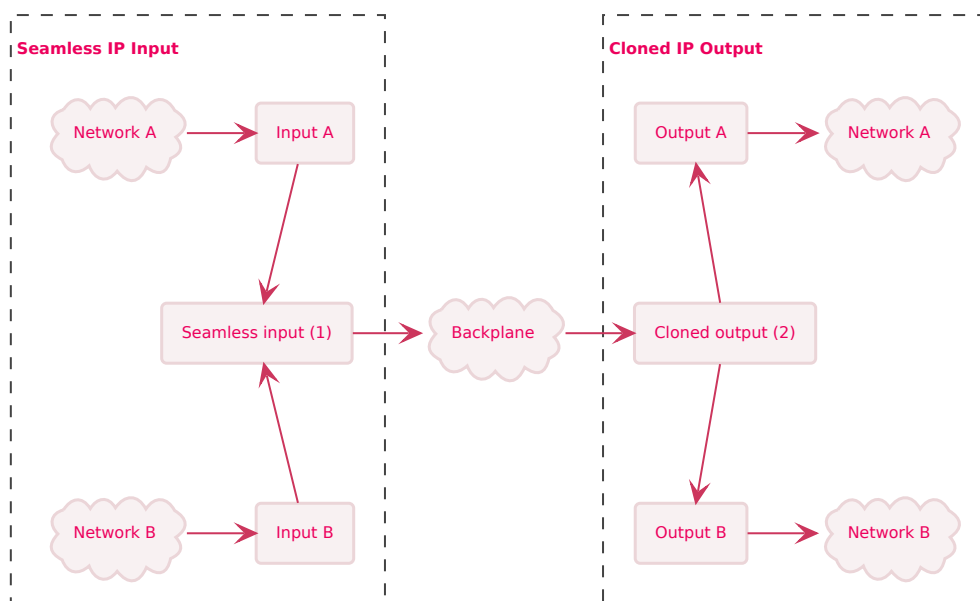


The `GetOutputs` and `SetOutputs` are symmetrical, which means the same data structure can be used in both. This means it is possible to run the `GetOutputs` command, make some changes and send it back with `SetOutputs`.

6.4. Seamless inputs and Cloned outputs

The IP gateway module support both Seamless IP inputs and generating cloned IP outputs. The details of these modes can be found in the user manual. To monitor or configure these modes using the JSON-RPC API, it is important to know the basic building blocks.

The diagram below shows the content flow of Seamless input and cloned output. An IP input is configured using the `SetInputs` command as described earlier. The `InputUdp` is a variant which supports `seamless`, and will configure seamless input if specified. It is needed to specify `InputUdpSettings` for A and B, each using IP interfaces which are defined on separate physical ports. The board will receive both streams and seamlessly merge the content of both. This merge point is the Input to the system, as described in [Sources](#). All TS processing and status will be a reflection of the merged IP stream. There are some status available for A and B in `GetIpInputStatus`.



- ① Seamless input. The content is a merge of A and B. The resulting stream is analyzed and processed like a normal IP input.
- ② Cloned output. The output stream is cloned and sent to network A and B.

Cloned output makes it possible to stream two identical IP streams into different networks, where downstream seamless capable devices can merge them. The stream is cloned after all output processing is complete. A cloned output is defined by `cloned` is specified in the `OutputUdp` variant.

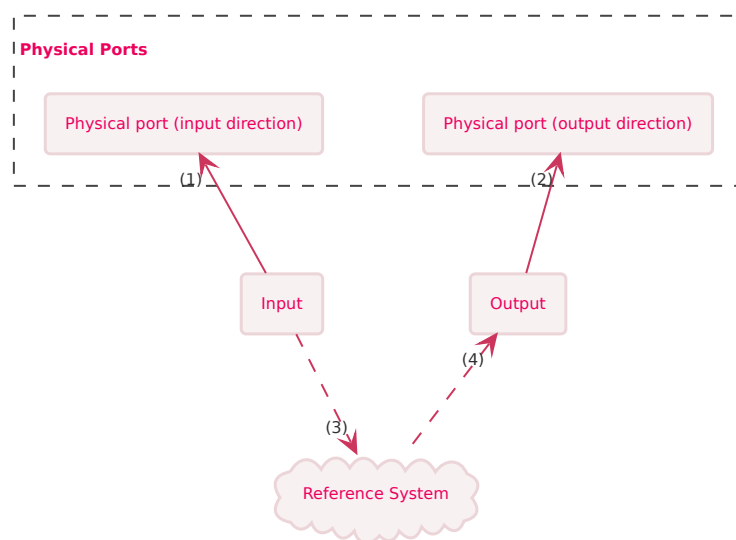
7. ASI

The ASI module offers both input and output functionality on 8 or 16 ports, depending on the hardware. Each port can be configured to either input or output direction. Both input and output streams are linked to these physical ports. All configuration is done through the `board` endpoint and the `asi` interface.

For a basic configuration of an ASI module, following steps should be performed

1. Configure the direction for each physical port, by using the `asiModule`.
2. Configure possible inputs using `input module`.
3. Configure possible outputs using `output module`.

The conceptual model of the ASI module is illustrated below:



- ① An ASI input is linked to a physical port UUID, with direction set to input
- ② An ASI output is linked to a physical port UUID, with direction set to output
- ③ Input module will analyze the received data-stream and publish available sources to the reference system as described in [Sources](#)
- ④ An Output will address sources in the reference system to generate an output data stream.

7.1. ASI physical ports

Method `GetAsiPhysicalPorts` returns the current configuration, which then can be changed by using `SetAsiPhysicalPorts` method.

Request

```
{
  "jsonrpc": "2.0",
  "method": "{{PKG_NAME}}:{{API_VERSION}}/asiModule/GetAsiPhysicalPorts",
  "params": {},
  "id": "GetAsiPhysicalPorts"
}
```

Response

```
{
  "jsonrpc": "2.0",
  "id": "GetAsiPhysicalPorts",
  "result": {
    "data": [
      {
        "key": "d142df2b-9fe3-47e7-878a-3a34c7cee53b", ①
        "value": {
          "enabled": true,
          "portNumber": 0,
          "label": "",
          "direction": "INPUT", ②
        }
      },
      ...
      {
        "key": "5c1113ab-4d7e-46ad-b73b-0c426aa45d2d",
        "value": {
          "enabled": true,
          "portNumber": 7,
          "label": "",
          "direction": "OUTPUT"
        }
      }
    ]
  }
}
```

① UUID of the AsiPhysicalPort

② Port direction, can be either **INPUT** or **OUTPUT**

7.2. ASI inputs

ASI inputs are maintained through the **input module**, similar to the one for the IP Gateway module. Each physical port in input direction has a default input configuration. Instead of creating a new input, the current input configuration should be modified by using **GetInputs** and **SetInputs**.

The configuration is split into two parts; transport and analysis. Transport settings allow enabling input features such as test generator and traffic policing, while analysis configuration consists of settings describing how the data should be interpreted by the system.

Request

```
{
  "jsonrpc": "2.0",
  "method": "{{PKG_NAME}}:{{API_VERSION}}/input/GetInputs",
  "params": {},
  "id": "GetInputs"
}
```

Response

```
{
  "jsonrpc": "2.0",
  "id": "GetInputs",
  "result": {
    "data": [
      {
        "key": "d478d30f-dff5-438b-8afa-9117e6cfd64", ①
        "value": {
          "label": "",
          "enabled": true,
          "transportSettings": {
            "asi": {
              "input": {
                "single": {
                  "id": "d5d933aa-0363-4a2c-a136-c23e0f8f45c8", ②
                  "portId": "d142df2b-9fe3-47e7-878a-3a34c7cee53b", ③
                  "portNumber": 0,
                  ...
                }
              }
            }
          },
          "analyzeMode": {
            "value": {
              "dvbMode": {
                "dejitter": { }
              }
            }
          }
        }
      },
      ...
    ]
  }
}
```

① Input config UUID, not used outside **input module**

② Input UUID, used for TS mapping

③ Physical port UUID, see **ASI physical ports**

7.3. ASI outputs

ASI outputs are maintained through the **output module**, similar to the one for the IP Gateway module. Each physical port in output direction has a default output configuration. Instead of creating a new output, the current output configuration should be modified by using **GetOutputs** and **SetOutputs**. The configuration is split into two parts; **transportSettings** and **outputSettings**. For more information regarding supported output modes see **IP Gateway Output modes**.

Request

```
{
  "jsonrpc": "2.0",
```

```

"method": "{PKG_NAME}:{API_VERSION}/output/GetOutputs",
"params": {},
"id": "GetOutputs"
}

```

Response

```

{
  "jsonrpc": "2.0",
  "id": "GetOutputs",
  "result": {
    "data": [
      {
        "key": "d1ce1791-2a69-4b58-9f71-e6540e9352ac",
        "value": {
          "label": "",
          "enabled": true,
          "transportSettings": {
            "asi": {
              "output": {
                "single": {
                  "id": "5891147a-e8a0-45f7-be8a-a938a790a4f6",
                  "portId": "5cc578f3-7029-47ae-b32b-2a91f32a84da", ①
                  "portNumber": 6,
                  "byteMode": "ASI_BURST",
                  "packetSize": "ASI_PACKET_SIZE_188",
                  "tsrFlywheel": { ... }
                }
              }
            }
          },
          "outputSettings": {
            "tsWhitelistMode": { ②
              "dvbMode": {
                "orgNetworkId": {},
                "settings": { ... },
                "source": {
                  "multiplex": [ ③
                    {
                      "service": {
                        "source": "eb473d6b-6b56-58a3-8e18-7509c4304bbb",
                        "inputRedundancy": { ... },
                        "priority": 100,
                        "settings": { ... }
                      }
                    }
                  ]
                }
              }
            }
          }
        }
      }
    ]
  }
}

```

```
}
```

- ① Physical port UUID, see [ASI physical ports](#)
- ② Output mode, as described in IP Gateway [Output modes](#)
- ③ This output is in TsWhitelist dvb mode, with one service source defined. See [Sources](#) for more information.